Faith James

Dr. Huang

COSC 647

9 November 2026

Database Security in PostgreSQL: Auditing, Logging, Alerts, and Encryption

This report examines the free, open-source security features of PostgreSQL, focusing on the PostgreSQL Audit Extension (pgAudit). PostgreSQL was chosen because it is free, widely used, and runs on Windows, macOS, and Linux. It also covers all four areas named in the lab, tracing, auditing, alerts, and encryption, through a mix of built-in settings and free extensions.

Introduction to the Features

For tracing and auditing, PostgreSQL can log every connection and SQL statement, but this built-in output is unstructured text that is hard to feed into compliance tools (JusDB). pgAudit fills that gap by adding detailed session and object audit logging through PostgreSQL's standard logging facility (pgAudit, "pgaudit/pgaudit"). Session auditing records whole classes of statements, such as reads, writes, or schema changes, while object auditing records only activity on chosen tables. Each entry begins with AUDIT: and lists the statement class, command, object, and statement text, making the log easy to filter. Access is controlled through roles and row-level security, which lets a policy limit each user to their own rows (PostgreSQL Global Development Group, "Row Security"). For alerts, triggers can call NOTIFY so that any connected client running LISTEN receives a message the moment a sensitive value changes (PostgreSQL Global Development Group, "NOTIFY"). For encryption, the bundled pgcrypto extension encrypts columns and hashes passwords inside ordinary SQL, using AES-128 by default with AES-256 available (PostgreSQL Global Development Group, "F.26. pgcrypto"), while SSL/TLS protects data traveling over the network (PostgreSQL Global Development Group, "Secure TCP/IP Connections").

Test Environment

To explore these features, I installed PostgreSQL 16.15 on Ubuntu Linux 24.04 together with the free `postgresql-16-pgaudit` package, which provides pgAudit 16.0. pgAudit works only after it is listed in the `shared_preload_libraries` setting, so I added that line to `postgresql.conf`, turned on SSL with a self-signed certificate, and restarted the server. I then created a practice database named `labdb` and ran `CREATE EXTENSION` for both pgaudit and pgcrypto. All testing was done in psql, PostgreSQL's command-line client, and the server log was read directly from the data directory. Fig. 1 shows the installed extensions.

```
labdb=# \dx
                    List of installed extensions
   Name   | Version |   Schema   |           Description
----------+---------+------------+--------------------------------
 pgaudit  | 16.0    | public     | provides auditing functionality
 pgcrypto | 1.3     | public     | cryptographic functions
 plpgsql  | 1.0     | pg_catalog | PL/pgSQL procedural language
(3 rows)
```

Fig. 1. The \dx command listing the installed pgaudit and pgcrypto extensions.

Tracing and Auditing with pgAudit

I first tested session auditing by setting `pgaudit.log` to `write, ddl`, which records data changes and schema changes but not ordinary reads. Working as an *hr_admin* role, I created an *employees* table, inserted three employees, raised one salary, and then read the table. The log (fig. 2) received an AUDIT entry for the CREATE TABLE, the INSERT, and the UPDATE, but none for the SELECT, which shows that the class filter works as documented. One detail surprised me: the single CREATE TABLE statement produced four audit lines, because the serial column created a sequence and the primary key created an index, and pgAudit logs each object separately. Each line follows the same comma-separated layout of audit type, statement ID, class, command, object type, object name, and statement text. The final field reads `<not logged>` because statement parameters are not recorded unless `pgaudit.log_parameter` is turned on (pgAudit, "pgaudit/pgaudit").

```
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE SEQUENCE,SEQUENCE,public.employees_id_seq,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE TABLE,TABLE,public.employees,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,CREATE INDEX,INDEX,public.employees_pkey,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.524 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,1,1,DDL,ALTER SEQUENCE,SEQUENCE,public.employees_id_seq,"CREATE TABLE employees (id serial PRIMARY KEY, name text, dept text, salary numeric, owner_name text);",<not logged>
2026-11-09 00:26:57.525 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,2,1,WRITE,INSERT,TABLE,public.employees,"INSERT INTO employees (name, dept, salary, owner_name) VALUES ('Alice Chen','Sales',62000,'alice'), ('Bob Diaz','IT',71000,'bob'), ('Carla Ruiz','HR',58000,'carla');",<not logged>
2026-11-09 00:26:57.526 UTC [1082] postgres@labdb LOG:  AUDIT: SESSION,3,1,WRITE,UPDATE,TABLE,public.employees,UPDATE employees SET salary = 75000 WHERE name = 'Bob Diaz';,<not logged>
```

Fig. 2. Session auditing: AUDIT entries written to the server log for the CREATE TABLE, INSERT, and UPDATE run as hr_admin. The SELECT does not appear because the READ class was not enabled.

Next I tested object auditing, which is designed to watch specific data rather than every statement. I created a role named *auditor* that cannot log in, set `pgaudit.role` to that role, and granted it SELECT on only the *salary* column. With session logging reduced to schema changes, a query of names and departments left nothing in the log, while a query that read salaries produced an OBJECT audit entry (fig. 3). Later, when salaries were updated during the alert test, those updates were also recorded as object audit entries. This targeted approach is a practical way to keep audit logs focused on sensitive columns.

```
labdb=# SELECT name, salary FROM employees WHERE dept = 'IT';
   name   | salary
----------+--------
 Bob Diaz |  75000
(1 row)
```

```
2026-11-09 00:26:59.101 UTC [1090] postgres@labdb LOG:  AUDIT: OBJECT,1,1,READ,SELECT,TABLE,public.employees,"SELECT name, salary FROM employees WHERE dept = 'IT';",<not logged>
```

Fig. 3. Object auditing: the query that reads the audited salary column (top) and the OBJECT audit entry it produced (bottom).

Access Control with Row-Level Security

To limit which rows each user may see, I enabled row-level security on the employees table and created a policy named *own_row* that returns only rows whose *owner_name* matches the current user. When I switched to the *alice* role, the same SELECT statement returned only Alice's record, and as *bob* it returned only Bob's (fig. 4). No change to the query itself was needed, because the database applies the policy automatically. The documentation warns that

superusers and table owners bypass these policies unless the table is set to force row security, so policies protect ordinary application accounts rather than administrators (PostgreSQL Global Development Group, "Row Security").

```
labdb=# SET ROLE alice;
SET
labdb=# SELECT name, dept, salary FROM employees;
    name    | dept  | salary
------------+-------+--------
 Alice Chen | Sales |  62000
(1 row)

labdb=# SET ROLE bob;
SET
labdb=# SELECT name, dept, salary FROM employees;
   name   | dept | salary
----------+------+--------
 Bob Diaz | IT   |  75000
(1 row)
```

Fig. 4. Row-level security: the same query returns only Alice's row for alice and only Bob's row for bob.

Alerts with Triggers and LISTEN/NOTIFY

PostgreSQL has no built-in alert console, so I built a simple alert with a trigger. I wrote a PL/pgSQL trigger function that runs after any salary update and calls `pg_notify` on a channel named *security_alerts* whenever a salary rises by more than ten percent.

To test the alert, I opened two sessions side by side. The first session acted as a security monitor by running LISTEN, and the second, logged in as hr_admin, made two changes. Carla Ruiz's small raise from 58,000 to 59,000 did not trigger an alert, but Alice Chen's jump from 62,000 to 90,000 sent a message to the monitor (fig. 5) that named the employee, the old and new values, and the user who made the change. In psql the notification appears after the monitor's next command, whereas a real monitoring program would use a client library that waits for notifications continuously.

```
Asynchronous notification "security_alerts" with payload "ALERT: Alice Chen salary changed 62000 -> 90000 by hr_ad
min" received from server process with PID 1110.
```

Fig. 5. The alert received by the monitoring session after hr_admin raised Alice Chen's salary from 62,000 to 90,000.

Encryption with pgcrypto and SSL/TLS

To test column encryption, I stored a sample Social Security number with `pgp_sym_encrypt` using AES-256. As fig. 6 shows, the stored value is unreadable binary data, the correct key returns the original number, and a wrong key fails with "Wrong key or corrupt data." I also stored a password with `crypt` and a Blowfish (bcrypt) salt, which produces a one-way hash; the login check returned true for the correct password and false for a guess. These functions were easy to use because they work inside normal INSERT and SELECT statements. However, the encryption key had to be typed directly into the SQL, which means it can appear in logs or application code, and this is why key management is left as the user's responsibility (Neon).

```
labdb=# SELECT name, left(encode(ssn_enc, 'hex'), 40) || '...' AS ssn_as_stored FROM customers;
   name   |                ssn_as_stored
----------+-----------------------------------------
 Dana Lee | c30d04090302aa0b0e0279d18eba7fd23c0151cf...
(1 row)

labdb=# SELECT name, pgp_sym_decrypt(ssn_enc, 'S3cretKey!') AS ssn FROM customers;
   name   |     ssn
----------+-------------
 Dana Lee | 123-45-6789
(1 row)

labdb=# SELECT pgp_sym_decrypt(ssn_enc, 'WrongKey') FROM customers;
psql:7_crypto.sql:5: ERROR:  Wrong key or corrupt data
```

```
labdb=# SELECT username, pw_hash FROM app_users;
 username |                           pw_hash
----------+--------------------------------------------------------------
 dana     | $2a$08$EBqpkBfG56FH2KHlQLIPpOF0jV2Sx38UEZ1YG3gEQFgbR3Mgby6Oa
(1 row)

labdb=# SELECT username, pw_hash = crypt('Tr0ub4dor', pw_hash) AS login_ok FROM app_users;
 username | login_ok
----------+----------
 dana     | t
(1 row)

labdb=# SELECT username, pw_hash = crypt('guess123', pw_hash) AS login_ok FROM app_users;
 username | login_ok
----------+----------
 dana     | f
(1 row)
```

Fig. 6. pgcrypto: the encrypted SSN as stored, decrypted with the correct key, and rejected with a wrong key (top); a bcrypt password hash and login checks (bottom).

Encryption in transit was confirmed by connecting with `sslmode=require`. Both psql's `\conninfo` command and the `pg_stat_ssl` view showed that the session used TLS 1.3 with a 256-bit AES-GCM cipher (fig. 7).

```
labdb=# \conninfo
You are connected to database "labdb" as user "postgres" on host "localhost" (address "127.0.0.1") at port "5433".
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
labdb=# SELECT pid, ssl, version, cipher, bits FROM pg_stat_ssl WHERE pid = pg_backend_pid();
 pid  | ssl | version |         cipher         | bits
------+-----+---------+------------------------+------
 1116 | t   | TLSv1.3 | TLS_AES_256_GCM_SHA384 |  256
(1 row)
```

Fig. 7. TLS 1.3 connection details reported by \conninfo and the pg_stat_ssl view.

User Experience

The user experience is powerful but demanding. Nearly everything is configured through SQL commands and text files, and there is no built-in graphical console for reviewing audits or alerts. pgAudit has the steepest learning curve because it must be preloaded and the server restarted, although cloud platforms such as Supabase reduce this to a dashboard toggle (Supabase). After that first restart, I could change audit settings with `ALTER SYSTEM` and `pg_reload_conf()` without restarting again, which made experimenting much easier. pgAudit's fixed log format is far easier to read than the default log, but broad settings produce so much output that important events can get buried, as the four lines from one CREATE TABLE showed, and AWS advises monitoring storage when it is enabled (Amazon Web Services). pgcrypto uses familiar SQL syntax but leaves key management to the user, and keys stored beside the data defeat the purpose (Neon).

Pros and Cons

The main strengths of these tools are cost and depth. pgAudit also keeps cleartext passwords out of the log (Amazon Web Services), a risk the built-in logging carries (Microsoft). I confirmed this in my own test: with the built-in `log_statement` setting turned on, a password change was written to the log in plain text, but with pgAudit logging the same kind of command, the password was replaced by `<REDACTED>` (fig. 8).

```
2026-11-09 00:27:05.411 UTC [1124] postgres@labdb LOG:  statement: ALTER ROLE hr_admin PASSWORD 'N3wSecret!';
2026-11-09 00:27:06.991 UTC [1132] postgres@labdb LOG:  AUDIT: SESSION,1,1,ROLE,ALTER ROLE,,,ALTER ROLE hr_admin P
ASSWORD <REDACTED>,<not logged>
```

Fig. 8. The same ALTER ROLE command logged by built-in statement logging (password visible) and by pgAudit (password redacted).

Advantages observed in this lab include:

- Every feature tested is free, production-grade, and supported by major cloud providers such as AWS, Azure, and Supabase.
- Audit entries have a consistent, filterable format, and session and object auditing can be combined to control detail.
- pgAudit redacts passwords that the built-in logger records in plain text.
- Row-level security, NOTIFY, pgcrypto, and TLS 1.3 are included with PostgreSQL at no extra cost.

The weaknesses are complexity and gaps. Unlike commercial products that bundle auditing, alerting, and reporting in one console, PostgreSQL users must assemble these pieces themselves:

- Setup requires editing configuration files and restarting the server, and there is no graphical interface for audits or alerts.
- Broad audit settings generate a large volume of log data that must be stored and reviewed.
- pgAudit cannot reliably audit superusers, and because logging is best-effort, a committed transaction may lack an audit entry after a crash (pgAudit, "pgaudit/pgaudit").
- LISTEN/NOTIFY reaches only clients connected at that moment and cannot send email or text messages by itself.
- Encrypted columns cannot be searched efficiently, and encryption keys must be managed outside the database.

Conclusion

In conclusion, PostgreSQL combined with pgAudit and pgcrypto offers a complete, free set of database security controls. In this lab, pgAudit recorded exactly the classes of activity it was configured for, row-level security hid other users' rows, a trigger raised a real-time alert on a suspicious salary change, and pgcrypto and TLS protected data at rest and in transit. These tools suit organizations willing to invest setup time, especially those with compliance needs, provided

audit log volume is managed and logs are shipped to a separate system. For beginners, the lack of a graphical interface is the biggest barrier.

Works Cited

Amazon Web Services. "Using pgAudit to Log Database Activity." *Amazon RDS User Guide*, docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Appendix.PostgreSQL.CommonDBATasks.pgaudit.html. Accessed 8 Nov. 2026.

JusDB. "pgAudit: PostgreSQL Audit Logging for SOC2, PCI-DSS, and HIPAA Compliance." *JusDB*, 5 Mar. 2026, www.jusdb.com/blog/pgaudit-postgresql-audit-logging-compliance.

Microsoft. "Audit Logging in Azure Database for PostgreSQL Flexible Server." *Microsoft Learn*, 16 July 2026, learn.microsoft.com/en-us/azure/postgresql/security/security-audit.

Neon. "The pgcrypto Extension." *Neon Docs*, 5 June 2026, neon.com/docs/extensions/pgcrypto.

pgAudit. "pgaudit/pgaudit: PostgreSQL Audit Extension." *GitHub*, github.com/pgaudit/pgaudit. Accessed 8 Nov. 2026.

---. "PostgreSQL Auditing Extension." *pgAudit*, www.pgaudit.org. Accessed 8 Nov. 2026.

PostgreSQL Global Development Group. "F.26. pgcrypto—Cryptographic Functions." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/pgcrypto.html. Accessed 8 Nov. 2026.

---. "NOTIFY." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/sql-notify.html. Accessed 8 Nov. 2026.

---. "Row Security Policies." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/ddl-rowsecurity.html. Accessed 8 Nov. 2026.

---. "Secure TCP/IP Connections with SSL." *PostgreSQL 18 Documentation*, www.postgresql.org/docs/current/ssl-tcp.html. Accessed 9 Nov. 2026.

Supabase. "PGAudit: Postgres Auditing." *Supabase Docs*, 5 Oct. 2026, supabase.com/docs/guides/database/extensions/pgaudit.